Returns values from the newly bound array

Returns values from the newly bound array, because we used a subselect

```
d3.selectAll("g").each(function(d) {console.log(d)});
d3.selectAll("text").each(function(d) {console.log(d)});
d3.selectAll("circle").each(function(d) {console.log(d)});
```

Returns values from the old tweetData array, because we haven't specified overwriting with a subselect

The `.exit()` function isn't intended to be used for binding a new array of completely different values like this. Instead, it's meant to update the page based on the removal of elements from the array that's been bound to the selection. But if you plan to do this, you need to specify how the `.data()` function binds data to your selected elements. By default, `.data()` binds based on the array position of the data value. This means, in the previous example, that the first four elements in our selection are maintained and bound to the new data, while the rest are subject to the `.exit()` function. In general, though, you don't want to rely on array position as your binding key. Rather, you should use something meaningful, such as the value of the data object itself. The key requires a string or number, so if you pass a JSON object without using JSON.stringify, it treats all objects as `"[object object]"` and only returns one unique value. To manually set the binding key, we use the second setting in the `.data()` function and use the inline syntax typical in D3.

Listing 2.7 Setting the key value in data-binding

```
function dataViz(incomingData) {

incomingData.forEach(function(el) {
     el.impact = el.favorites.length + el.retweets.length;
     el.tweetTime = new Date(el.timestamp);
})

var maxImpact = d3.max(incomingData, function(el) {
     return el.impact
});

var startEnd = d3.extent(incomingData, function(el) {
     return el.tweetTime
});

var timeRamp = d3.time.scale().domain(startEnd).range([ 50, 450 ]);
var yScale = d3.scale.linear().domain([ 0, maxImpact ]).range([ 0, 460 ]);
var radiusScale = d3.scale.linear()
     .domain([ 0, maxImpact ])
     .range([ 1, 20 ]);

d3.select("svg").selectAll("circle")
.data(incomingData, function(d) {
     return JSON.stringify(d)
}).enter().append("circle").attr("r", function(d) {
     return radiusScale(d.impact)
```

We could use any unique attribute as the key, but using the entire object works if we don't have a unique value, though we have to stringify it first.